

Department of Artificial Intelligence and Machine Learning

AIML · 7th Semester

PROJECT REPORT

WasteVision AI

Waste Recognition, Sorting Guidance, and Cleanup Prioritization

Project Title	WasteVision AI
Domain	Environmental sustainability and waste management
Subject	Image Analytics using Computer Vision AML23701

Submitted by

Guide: Prof. Vijaya Dalawai

Team Member	Name	USN
1	Aditi S Yaranal	1GA23AI003
2	Adhisha Sreedith	1GA23AI002

Department of Artificial Intelligence and Machine Learning · 7th Semester

Table of Contents

1. Project Overview

2. Input Data and Model Preparation

3. System Architecture

4. Implementation Details

5. Test Sheet: Sample Scenarios and Outputs

6. Evaluation

7. Failure Case Analysis

8. Limitations and Future Scope

9. Learnings and Reflection

10. Use of AI Tools Disclosure

Appendix A. Interface Evidence

1. Project Overview

Project title: WasteVision AI

Domain: Environmental sustainability and community waste response

Target users: Residents, campus communities, and cleanup coordinators

Problem statement: People may be unsure how to sort individual items, while mixed waste and overflowing public bins can be difficult to report and prioritize. A single workflow can connect visual inspection, plain-language sorting guidance, and a trackable cleanup queue.

Value of the solution: The app combines photo-based waste scanning, free-text disposal lookup, hotspot reporting, a transparent triage estimate, and an impact dashboard. The prototype helps organize next steps; it does not establish official safety or environmental findings.

2. Input Data and Model Preparation

Item	Details
Input sources	User-uploaded JPG/JPEG/PNG photos; typed waste descriptions; manually entered hotspot location and conditions.
Image model	YOLO-World open-vocabulary adapter with default yolov8s-worldv2.pt; Roboflow adapter is also supported by configuration.
Waste labels	Common bottles, bags, wrappers, cardboard, cans, glass, food scraps, and visible litter; labels normalized through config.py.
Normalized categories	General, plastic, paper, organic, metal, and glass. Unsupported labels are filtered.
Image preparation	Image upload is passed to the detector; inference uses a larger image size and test-time augmentation in the Ultralytics adapter.
Text preparation	Typed item text is normalized and matched against explicit category and alias rules.
Persisted records	SQLite stores report details and scan counts; the app seeds clearly marked illustrative demo records on first run.
Data caveat	No labeled training/evaluation corpus or measured dataset size is supplied. Demo records and detections are not ground truth.

This project does not use a document corpus, text chunking, or retrieval-augmented generation. Inputs are uploaded images, typed item descriptions, and user-entered hotspot details. The scanner maps model labels to a small set of waste categories; sorting guidance uses local rule-based mappings.

3. System Architecture

System flow: User image or item description → Streamlit page → model label mapping or sorting rules → disposal guidance; hotspot details → priority estimate → SQLite report queue → Plotly impact dashboard.

WasteVision AI connects recognition and community action in one Streamlit workflow.

3.1 Pipeline Stages

Stage	What the system does	Tool / library	Key design choice
Scan	Find candidate objects in an uploaded image.	Ultralytics YOLO-World; optional Roboflow inference SDK	Open-vocabulary prompts broaden visible object search, but are not a waste-specific validated model.
Normalize	Map recognized labels to supported waste categories.	config.py label aliases and filters	Ignore unsupported labels to reduce arbitrary categories.
Guide	Match a typed item to a disposal stream and plain-language advice.	Rule-based recommender	Give a cautious fallback for unknown descriptions; local rules may differ.
Triage and review	Estimate hotspot priority and store/update report status.	Python priority logic; SQLite	Show severity, overflow, sensitive-site inputs, and score explanation; label score as an estimate.
Visualize	Summarize materials, urgency, status, trends, and hotspots.	Streamlit, pandas, Plotly	Separate demo data from field reports where the source filter is available.

3.2 Sorting Guidance Rules

No generative language-model prompt is used by the application. Typed item descriptions are matched to explicit local waste rules. The scanner uses open-vocabulary object prompts and then maps supported labels to normalized waste categories.

Example lookup: “eggshells” → Wet waste / compost. Guidance: put food scraps in the wet-waste or compost stream and keep plastic packaging out. The app advises checking local rules for uncertain materials.

3.3 Handling Unknown Items

Unknown typed descriptions receive a clearly labeled residual-waste fallback rather than an invented specific route. Unknown model labels are ignored until they are mapped in config.py. Hazardous or uncertain items should be checked with the local waste authority.

4. Implementation Details

4.1 Tools and Libraries

Tool / library	Purpose
Streamlit	Multipage-style interactive application and controls.
Ultralytics YOLO-World	Default open-vocabulary image detection adapter.
Roboflow inference-sdk	Optional hosted model provider.

Tool / library	Purpose
Pillow and OpenCV	Image handling and model input support.
SQLite	Local persistence for reports and scan totals.
pandas and Plotly	Dashboard data shaping and interactive charts.

4.2 What We Built vs. What We Borrowed

Component	Built by team	Borrowed / adapted from
Waste workflow and Streamlit pages	Yes	Streamlit framework and widgets.
Label mapping and sorting rules	Yes	Waste-category conventions; local municipal rules need configuration.
Priority estimate and status tracking	Yes	Project-defined transparent triage factors; not an official standard.
Dashboard charts and report persistence	Yes	Plotly charts and Python SQLite library.

4.3 Report Data and Priority Logic

- Data records: Reports store location, categories, item count, severity, overflow and sensitive-site flags, priority estimate, status, source, and time.
- Persistence: SQLite stores reports and scan counts; the app seeds clearly labeled sample rows on first run.
- Priority logic: Combines item count, category variety, reporter-rated severity, overflow or dumping, and sensitive-site proximity. The result is a triage estimate, not an official hazard assessment.

5. Test Sheet: Sample Scenarios and Outputs

The table records five functional scenarios from the prototype and supplied screenshots. These are workflow observations, not a controlled accuracy study. A mixed-bin scan is marked partial because the example returned only one object.

#	Scenario	Observed output	Evidence / expected behavior	Result
1	Open the home page	Operations desk, four workflows, report and scan metrics.	Supplied home screenshot; navigation and actions are visible.	Pass
2	Scan the supplied mixed-bin photo	One “paper cup” detection at 49% confidence.	Scanner screenshot; many other objects are visible but were not reported.	Partial
3	Type “eggshells” in Smart Segregation	Wet waste / compost recommendation with handling advice.	Supplied segregation screenshot.	Pass

#	Scenario	Observed output	Evidence / expected behavior	Result
4	Enter a hotspot report with severity/site conditions	Report queue displays location, categories, priority inputs, and status.	Supplied report screenshot.	Pass
5	Review community impact	Material mix, urgency, response status, and report rhythm charts.	Supplied dashboard screenshot.	Pass

6. Evaluation

Measure	Result
Manual workflow scenarios observed	5
Workflow outcomes	4 passed; 1 partial
Scanner detections in supplied mixed-bin example	1 paper cup at 49% confidence
Detection precision / recall	Not measured; no labeled ground truth dataset supplied
Sorting lookup examples shown	1 (eggshells → wet waste / compost)
Priority score validation	Not measured; score is an internal triage estimate
Response time	Not measured

The supplied screenshots show the navigation, typed eggshell sorting guidance, hotspot intake and status queue, and dashboard views. The scanner ran on a mixed-bin photo but returned only one paper cup at 49% confidence. No labeled test dataset or ground-truth annotations were available, so model accuracy and detection recall were not measured.

7. Failure Case Analysis

The mixed-bin screenshot provides one observed weak case. The second case below is a known limitation of the current prototype, not a measured test result.

Field	Observed case: cluttered mixed bin	Known limitation: local sorting variation
Input	Uploaded photo of two crowded waste bins.	Typed or scanned material for which municipal rules vary.
Observed / possible output	Only one paper cup was returned, at 49% confidence.	A broad category route may not match local collection rules.
What went wrong	Visible mixed waste was largely missed in this example.	The prototype does not load municipality-specific rules.
Likely reason	Clutter, occlusion, small objects, prompt coverage, and non-waste-trained weights can affect detection.	Sorting mappings are intentionally broad and locally variable.
Possible fix	Use a labeled waste image set, improve prompts/model training, and evaluate precision and recall.	Add configurable local rules and a clear route to local authority guidance.

8. Limitations and Future Scope

Limitations: The open-vocabulary detector is not trained or validated specifically for waste. Small, hidden, overlapping, or unfamiliar objects may be missed or mislabeled; the screenshot shows only one detection in a cluttered bin. Sorting rules are broad and vary by municipality. The SQLite prototype has no authentication, moderation, or multi-user synchronization. Seeded reports and demo detections are illustrative, not verified observations. Priority is a triage estimate, not an official hazard assessment.

Future improvements: Collect and label diverse local waste images, then validate and fine-tune a waste-specific detector. Measure per-class precision and recall against a held-out dataset, improve clutter and small-object detection, and add calibrated confidence warnings. Add municipal-specific sorting rules, multilingual lookup, report photo review, map-based location capture, and verified before-and-after cleanup tracking.

9. Learnings and Reflection

The project combines computer vision, rule-based guidance, persistence, and dashboard design in one prototype. The mixed-bin scan demonstrates the challenge of detecting small and overlapping objects and the need to evaluate outputs against labeled examples. The modular Streamlit structure keeps scanning, segregation, reporting, and analytics independently extensible. The work connects software implementation with practical community waste workflows.

10. Use of AI Tools Disclosure

AI tools disclosure: ChatGPT was used to assist with code review, debugging, and report drafting. The application itself does not call a generative LLM; scanning uses an object-detection model and segregation uses explicit rules. Project repository: https://github.com/Aditisy/Waste-Vision-AI

Appendix A. Interface Evidence

The screenshots below document the prototype views and manual scenarios summarized in the report.

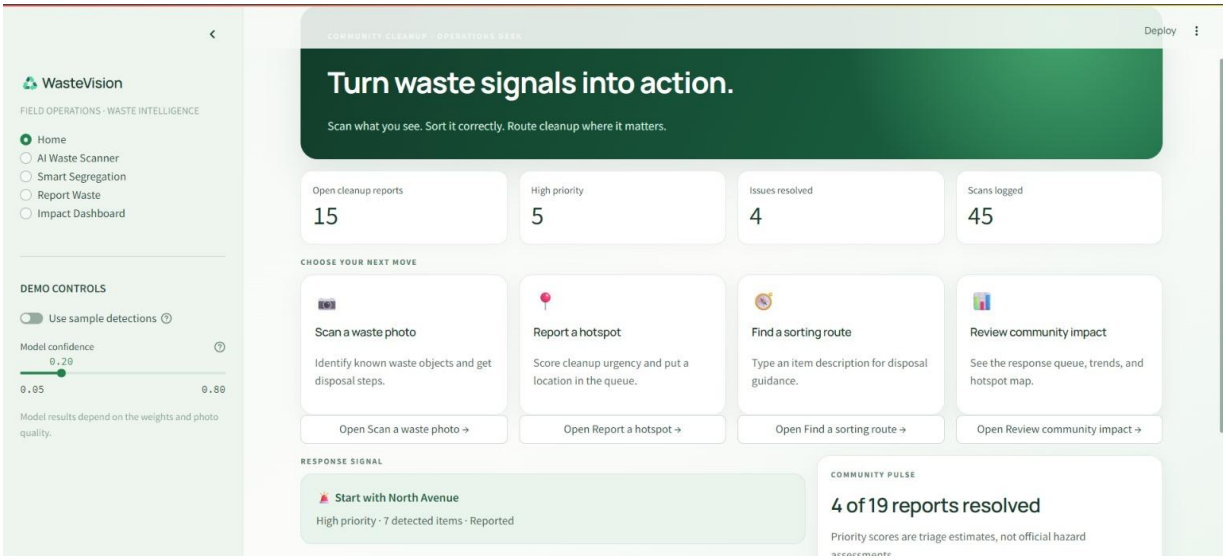


Figure 1. Home operations desk showing report, priority, resolution, and scan metrics.

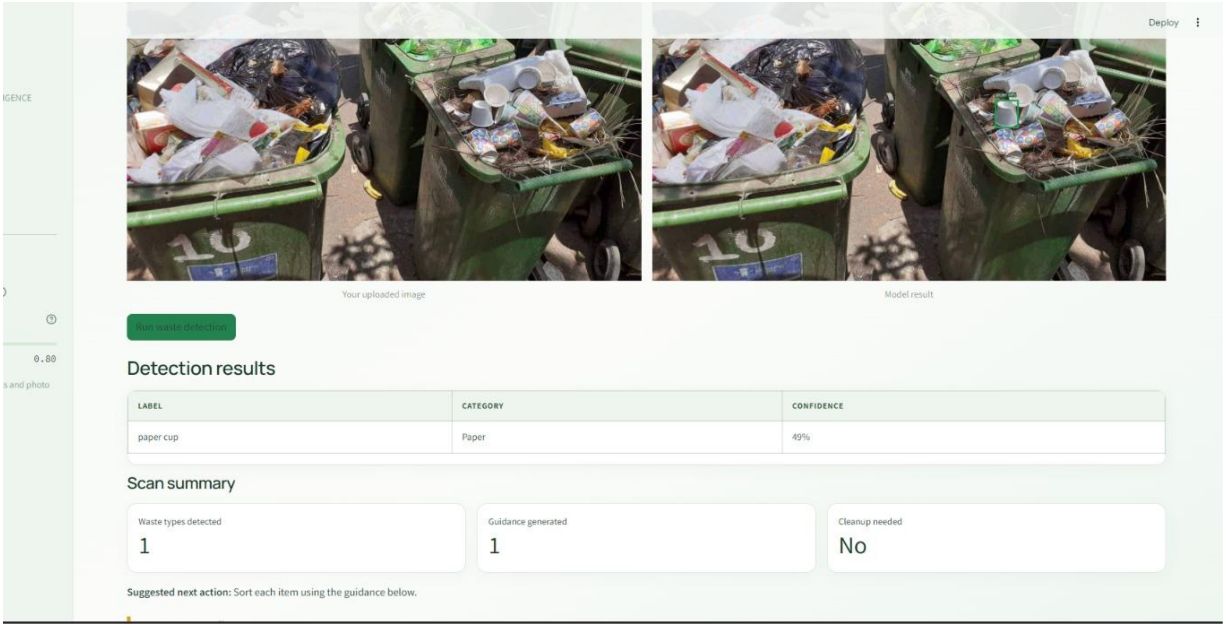


Figure 2. Mixed-bin scan example. The shown result detected one paper cup at 49% confidence, illustrating the current recognition limitation.

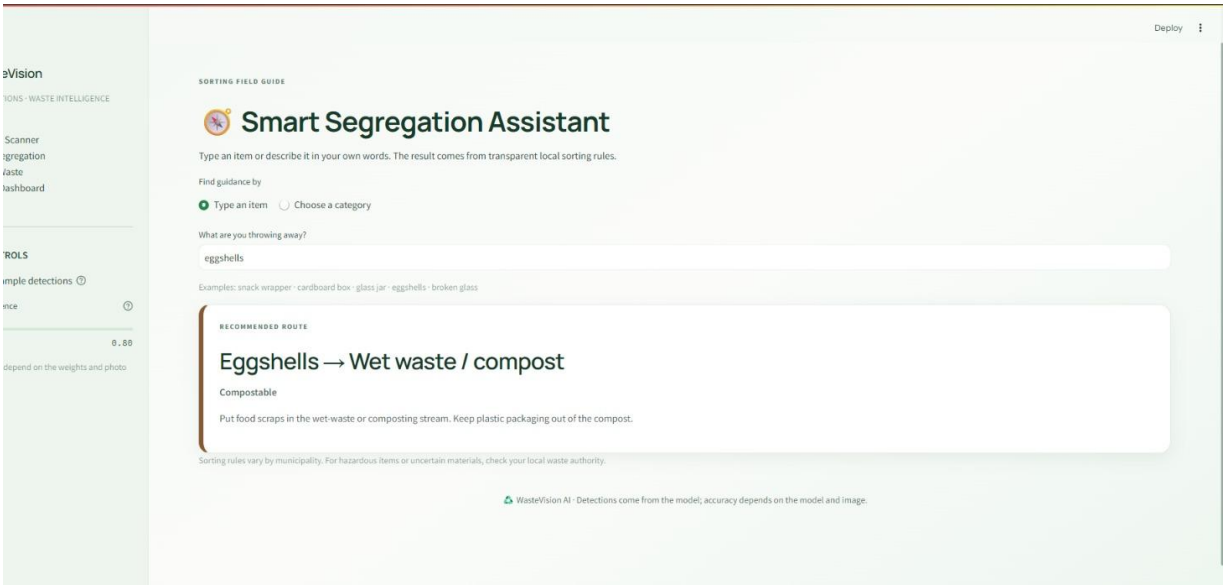


Figure 3. Free-text segregation lookup for "eggshells," routed to wet waste or compost with handling guidance.

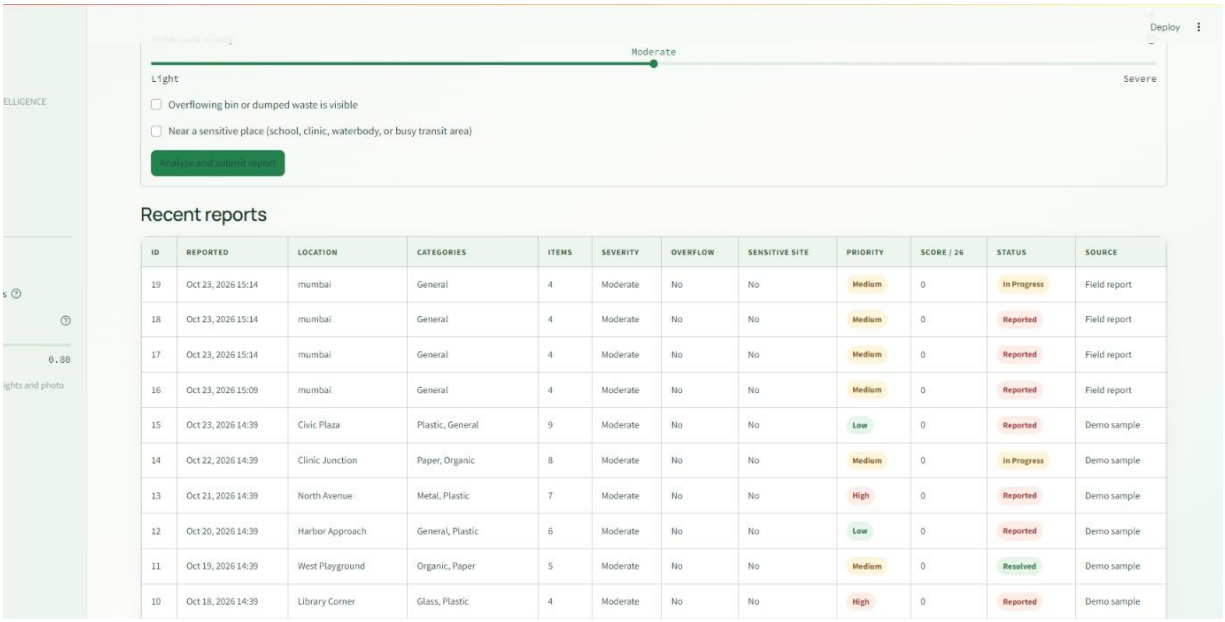


Figure 4. Hotspot report inputs and response queue, including visible severity, overflow, sensitive-site proximity, priority, and status.

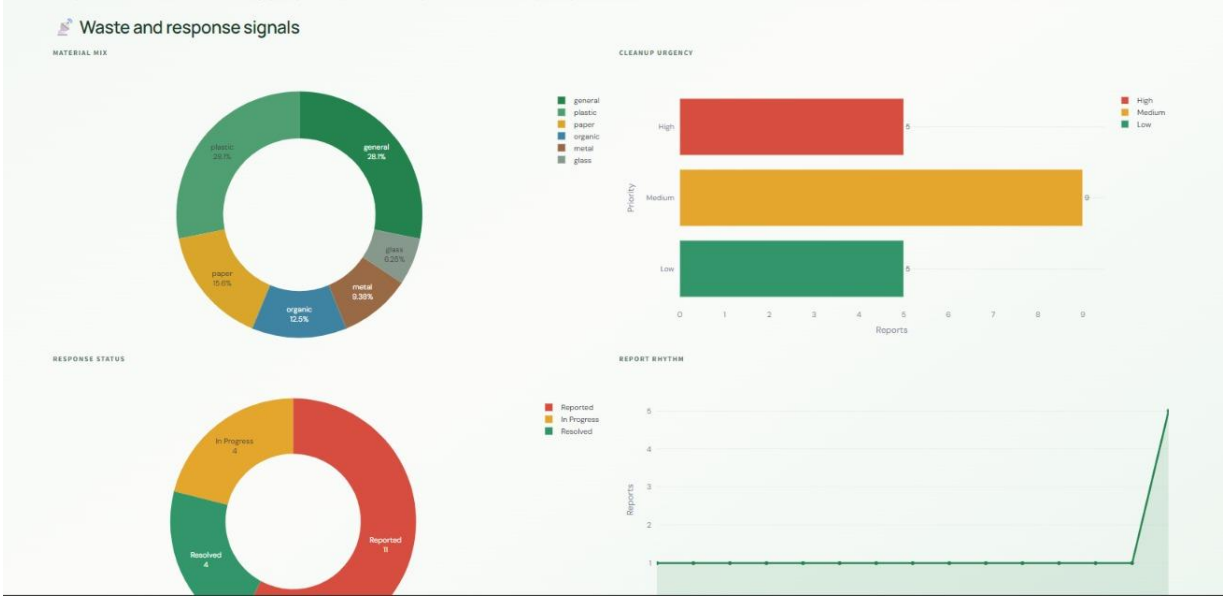


Figure 5. Impact dashboard charts for material mix, urgency, response status, and report rhythm.